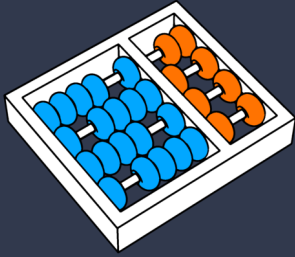




UNICAMP



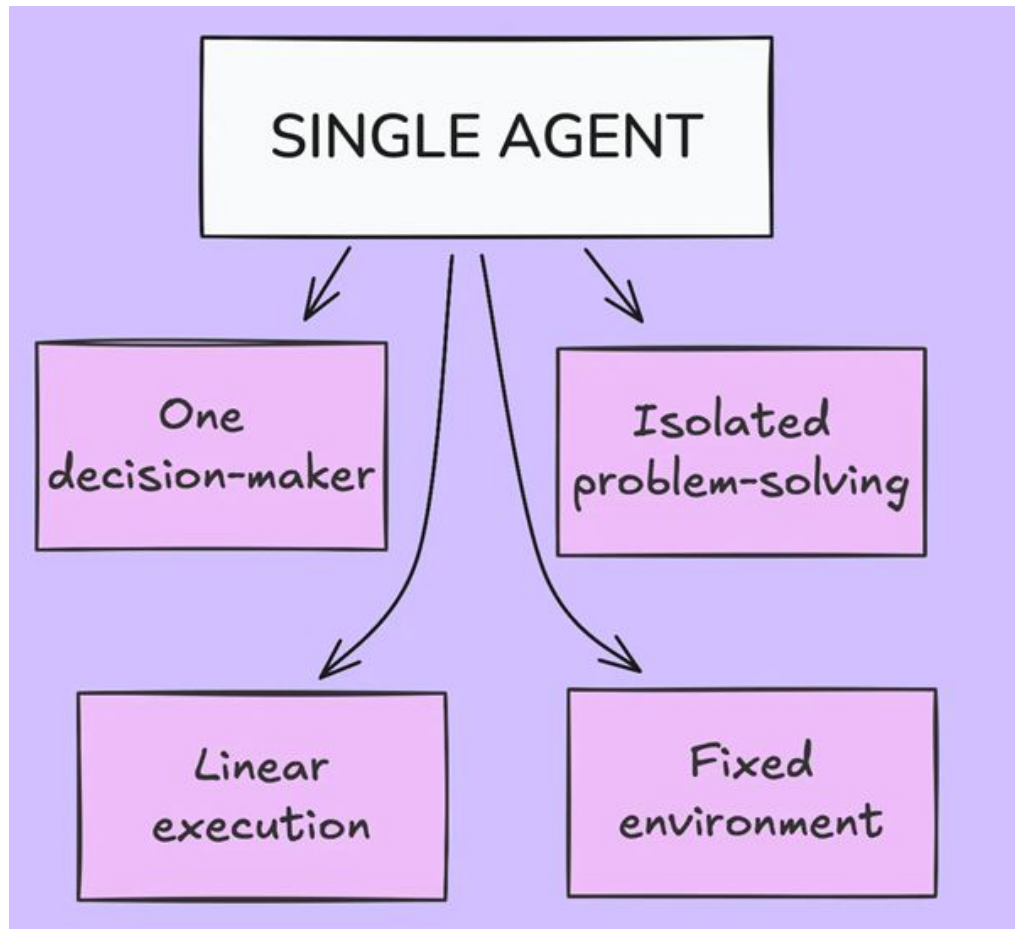
Arquitetura de Sistemas Multiagentes

Prof. Dr. Julio Cesar dos Reis
dosreis@unicamp.br

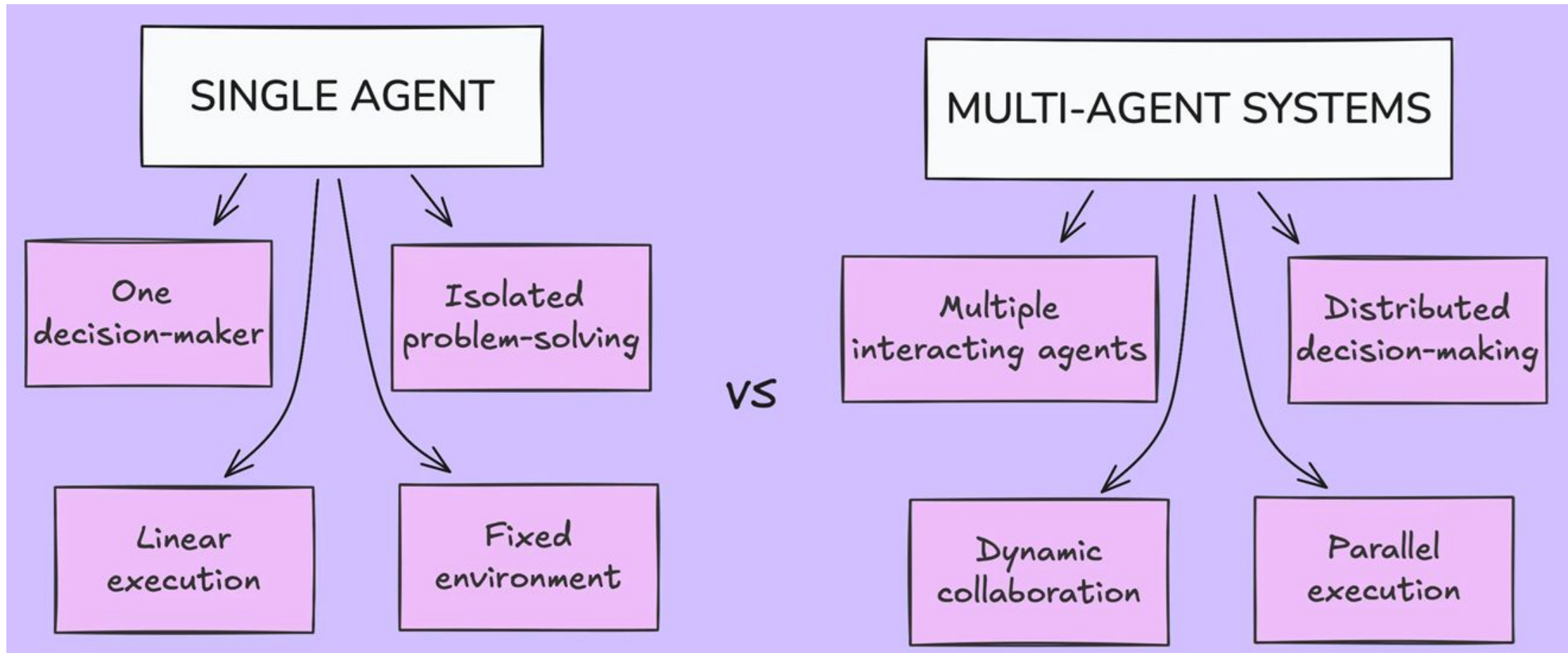
Agenda

- ❑ Motivação
- ❑ Sistema multiagente
- ❑ Modelos de autonomia dos Agentes
- ❑ Arquiteturas Multiagentes

Monoagente vs. Multiagente



Monoagente vs. Multiagente



Quando um único agente não é suficiente?

❏ Imagine um agente que precisa simultaneamente:

- pesquisar documentos
- navegar na Web
- analisar planilhas
- escrever código
- responder o usuário
- consultar bancos de dados
- gerar gráficos
- planejar tarefas futuras

Quando um único agente não é suficiente?

- ❏ Conforme novas capacidades são adicionadas:
 - aumenta o contexto
 - aumenta o número de ferramentas
 - aumenta o custo
 - piora o planejamento
 - cresce o risco de erros
- ❏ O problema deixa de ser o modelo LLM.
- ❏ O problema passa a ser a arquitetura.

Problema

- ❑ Desenvolver sistemas com agentes pode se tornar mais complexo ao longo do tempo, tornando-os mais difíceis de gerenciar e dimensionar
 - ❑ O agente tem muitas ferramentas à disposição
 - ❑ Podem tomar decisões ruins sobre qual ferramenta chamar em seguida
 - ❑ O contexto se torna muito complexo para um único agente acompanhar
 - ❑ Há necessidade de várias áreas de especialização no sistema
 - Ex: planejador, pesquisador, especialista em matemática, etc.

Evolução Arquitetural

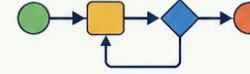
1. PROGRAMA TRADICIONAL



Lógica e dados centralizados.
Execução sequencial e determinística.

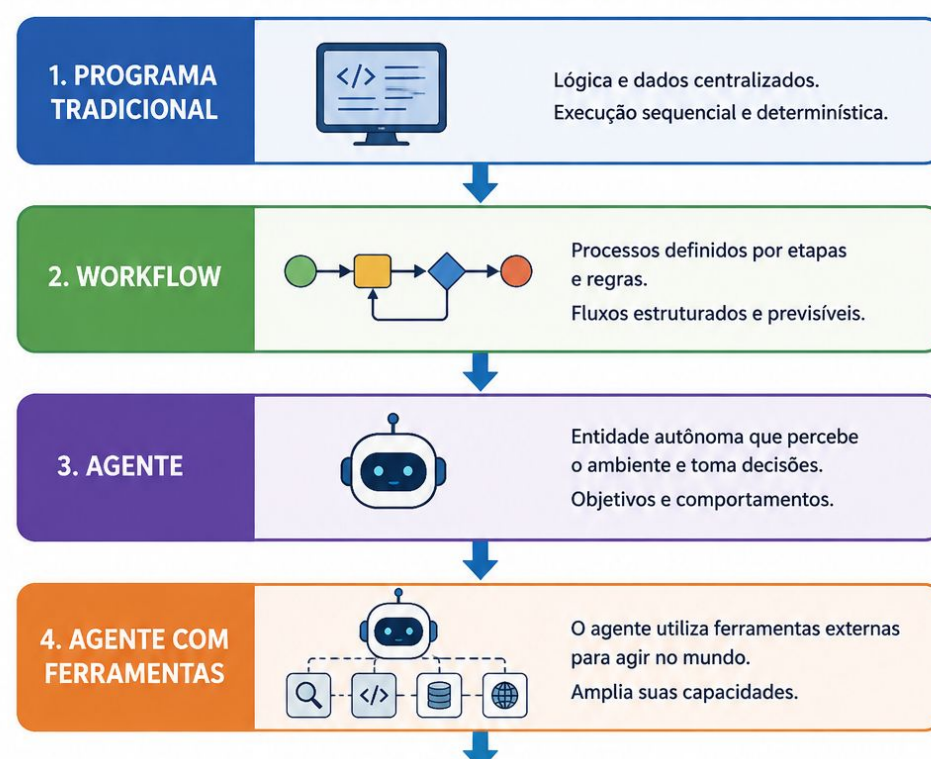


2. WORKFLOW

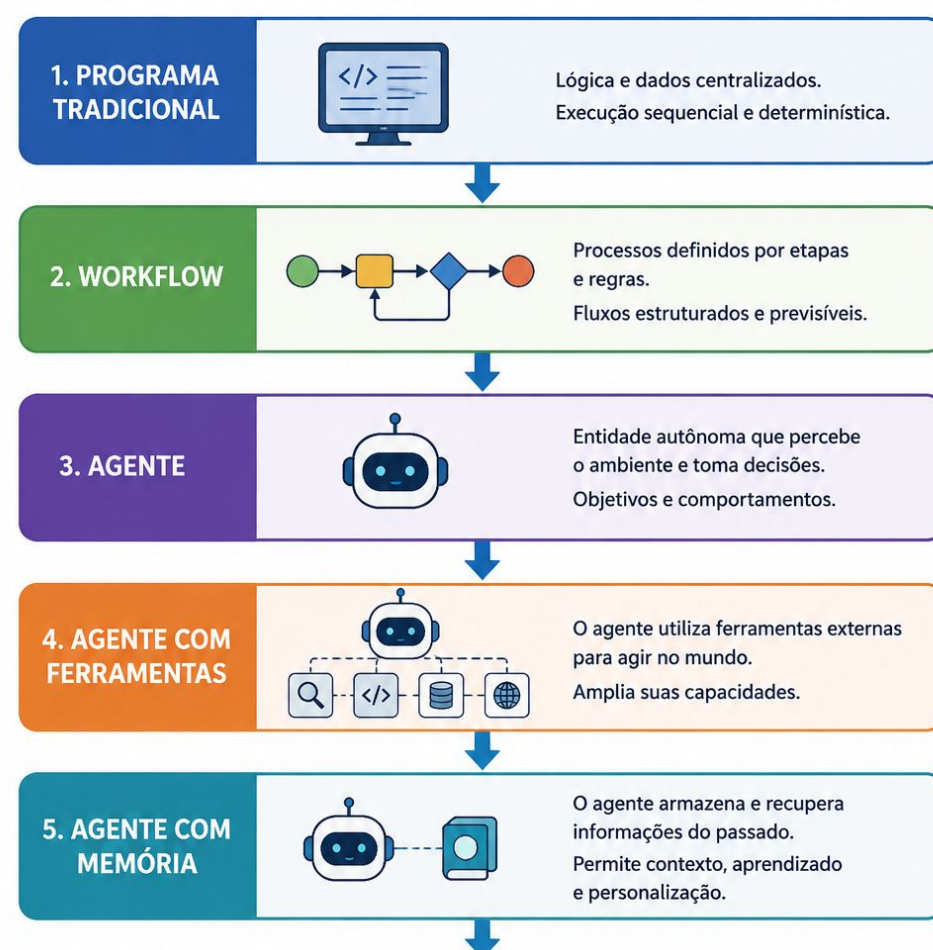


Processos definidos por etapas
e regras.
Fluxos estruturados e previsíveis.

Evolução Arquitetural



Evolução Arquitetural



Evolução Arquitetural



Evolução Arquitetural



Multiagentes

- ❑ Trabalham em equipe para resolver problemas complexos.
- ❑ Exemplo: Sistemas de negociação financeira autônoma.

Vantagem: Distribuem tarefas e otimizam resultados.

Desafios: Comunicação e coordenação precisam ser bem projetadas.

Sistemas Multiagentes

- ❑ Você pode considerar dividir sua solução em vários agentes menores
- ❑ Agentes independentes que compostos geram um **sistema multiagente**
 - ❑ Conjuntos de agentes que interagem para resolver problemas complexos.
 - ❑ Cada agente pode ter funções especializadas
 - ❑ Operar de forma autônoma

Benefícios de Sistemas multiagentes

- ❑ **Modularidade:** Facilita desenvolvimento e manutenção
- ❑ **Especialização:** Agentes podem focar em diferentes áreas
- ❑ **Escalabilidade:** Fácil expansão do sistema
- ❑ **Robustez:** O sistema continua funcionando mesmo que um agente falhe

Autonomia dos Agentes

- ❑ Um agente pode ser mais autônomo ou depender de um supervisor.

Modelos de Autonomia

Agentes Independentes

- Tomam decisões sem precisar de permissão externa.
- Exemplo: Carros autônomos que decidem quando frear.

Modelos de Autonomia

Agentes Independentes

- Tomam decisões sem precisar de permissão externa.
- Exemplo: Carros autônomos que decidem quando frear.

Agentes Supervisionados

- Precisam de validação antes de executar ações.
- Exemplo: Sistemas bancários que exigem aprovação para transações grandes.

Modelos de Autonomia

✓ Agentes Independentes

- Tomam decisões sem precisar de permissão externa.
- Exemplo: Carros autônomos que decidem quando frear.

✓ Agentes Supervisionados

- Precisam de validação antes de executar ações.
- Exemplo: Sistemas bancários que exigem aprovação para transações grandes.

✓ Agentes Cooperativos

- Trabalham em equipe para resolver um problema.
- Exemplo: Agentes que compartilham tarefas em um sistema de logística.

Autonomia não significa independência

- ❑ Autonomia de decisão
 - ❑ Pode decidir sozinho.
- ❑ Autonomia de execução
 - ❑ Pode executar ações.
- ❑ Autonomia de planejamento
 - ❑ Pode definir seus próprios objetivos.

Autonomia não significa independência

- ❑ Autonomia limitada
 - ❑ Sempre precisa de autorização.
 - ❑ Esse conceito é extremamente utilizado em Agentic AI.

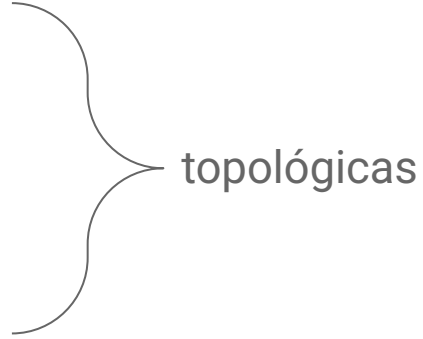
Especializar agentes

- ❏ Por que especializar agentes?
- ❏ Especialistas humanos trabalham melhor quando possuem funções bem definidas
 - O mesmo ocorre com agentes
- ❏ Cada agente possui
 - prompt próprio
 - ferramentas próprias
 - memória própria
 - objetivos próprios

Arquiteturas Multiagentes

- ❏ Definem como os agentes se comunicam e se organizam dentro do sistema
- ❏ 🚀 **Arquiteturas multiagentes** são fundamentais para sistemas de **IA** modernos!

Arquiteturas

1. **Arquitetura de Agente Único (*Single Agent*)**
 2. **Arquitetura em Rede (*Network*)**
 3. **Arquitetura com Supervisor (*Supervisor*)**
 4. **Arquitetura com Supervisor via *Tool-Calling***
 5. **Arquitetura Hierárquica (*Hierarchical*)**
 6. **Arquitetura Customizada (*Custom Workflow*)**
 7. **Arquitetura Star**
 8. **Arquitetura Ring**
 9. **Arquitetura Graph**
 10. **Arquitetura Bus**
 -
- 
- topológicas

Arquiteturas

1. **Arquitetura de Agente Único (*Single Agent*)**
2. **Arquitetura em Rede (*Network*)**
3. **Arquitetura com Supervisor (*Supervisor*)**
4. **Arquitetura com Supervisor via *Tool-Calling***
5. **Arquitetura Hierárquica (*Hierarchical*)**
6. **Arquitetura Customizada (*Custom Workflow*)**
7. **Arquitetura Star**
8. **Arquitetura Ring**
9. **Arquitetura Graph**
10. **Arquitetura Bus**
-

Qual delas é a melhor?

Depende da aplicação!

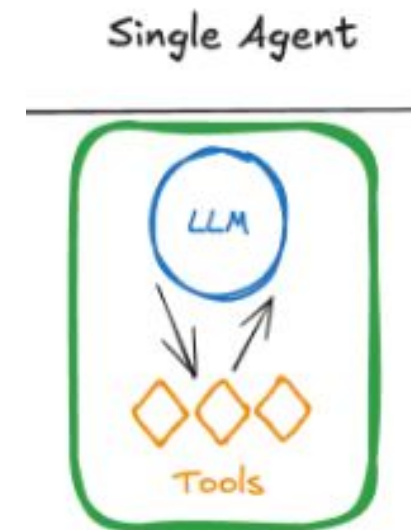
Vamos explorar cada uma em detalhes.

Arquitetura de Agente Único (*Single Agent*)

- ❑ Um único agente processa todas as entradas e toma todas as decisões.

Exemplo:

- ❑ Assistente virtual que responde perguntas do usuário sem interação com outros sistemas.



Arquitetura de Agente Único (*Single Agent*)

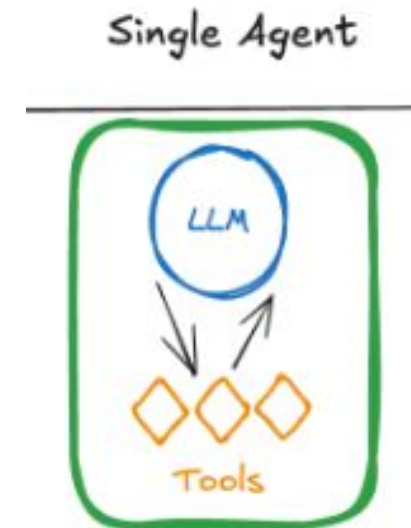
- ❑ Um único agente processa todas as entradas e toma todas as decisões.

Exemplo:

- ❑ Assistente virtual que responde perguntas do usuário sem interação com outros sistemas.

✓ **Vantagem:** Simples de implementar, gerenciar e baixo custo computacional.

✗ **Desafio:** Limitações em escalabilidade e pode ter dificuldades em lidar com múltiplas tarefas simultaneamente.

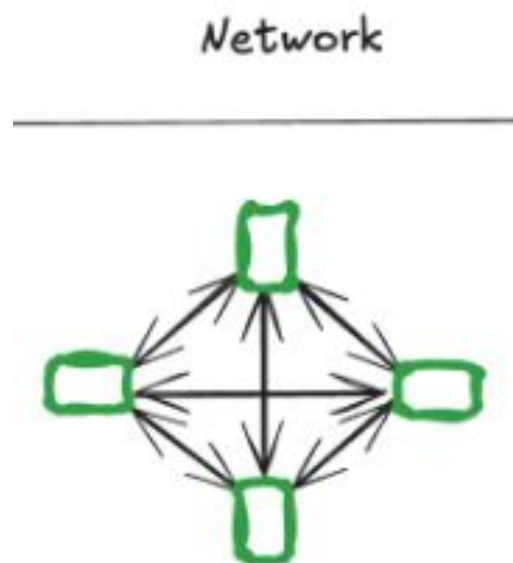


Arquitetura em Rede (*Network*)

- ❑ Cada agente pode se comunicar com todos os outros agentes.
- ❑ Não há uma hierarquia fixa ou sequência predeterminada.
- ❑ Útil para sistemas em que os agentes devem trabalhar de forma autônoma e decidir qual agente chamar em seguida.

Exemplo:

- ❑ Um sistema de pesquisa em que vários agentes analisam diferentes fontes de informação e compartilham resultados.



Arquitetura em Rede (*Network*)

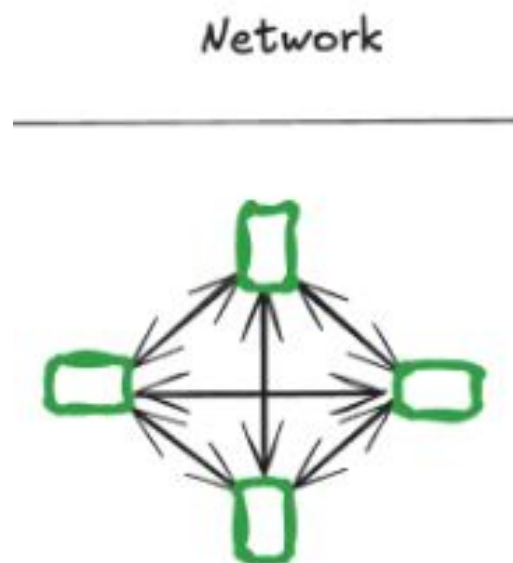
- ❑ Cada agente pode se comunicar com todos os outros agentes.
- ❑ Não há uma hierarquia fixa ou sequência predeterminada.
- ❑ Útil para sistemas em que os agentes devem trabalhar de forma autônoma e decidir qual agente chamar em seguida.

Exemplo:

- ❑ Um sistema de pesquisa em que vários agentes analisam diferentes fontes de informação e compartilham resultados.

✅ **Vantagem:** Alta flexibilidade e colaboração descentralizada.

❌ **Desafio:** Pode ser difícil coordenar um grande número de agentes.

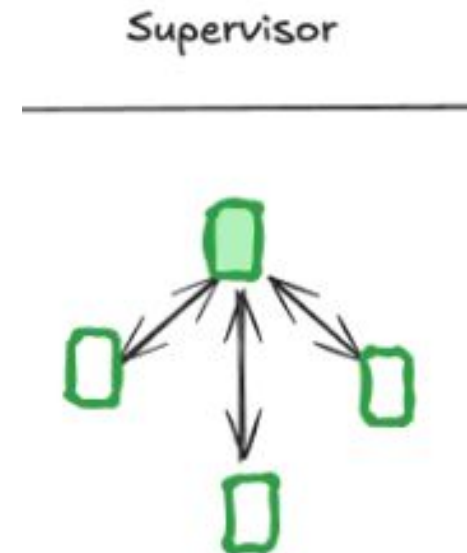


Arquitetura com Supervisor (*Supervisor*)

- ❑ Um agente supervisor gerencia quais agentes serão chamados e em que ordem.
- ❑ Cada agente executa sua tarefa e retorna o controle ao supervisor.
- ❑ Útil para sistemas que exigem coordenação centralizada.

Exemplo:

- ❑ Em um chatbot, o supervisor decide se a pergunta deve ser enviada para um agente de atendimento, um agente financeiro ou um agente técnico.



Arquitetura com Supervisor (*Supervisor*)

- ❑ Um agente supervisor gerencia quais agentes serão chamados e em que ordem.
- ❑ Cada agente executa sua tarefa e retorna o controle ao supervisor.
- ❑ Útil para sistemas que exigem coordenação centralizada.

Exemplo:

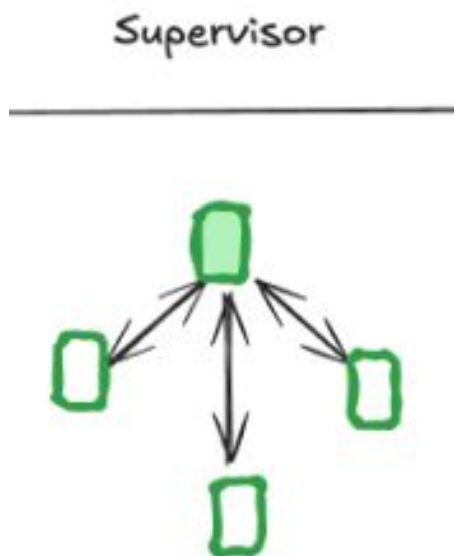
- ❑ Em um chatbot, o supervisor decide se a pergunta deve ser enviada para um agente de atendimento, um agente financeiro ou um agente técnico.

✅ **Vantagem:** Melhor controle sobre a execução.

❌ **Desafio:** Dependência do supervisor pode causar gargalos.

o supervisor nunca executa?

Ele apenas coordena?

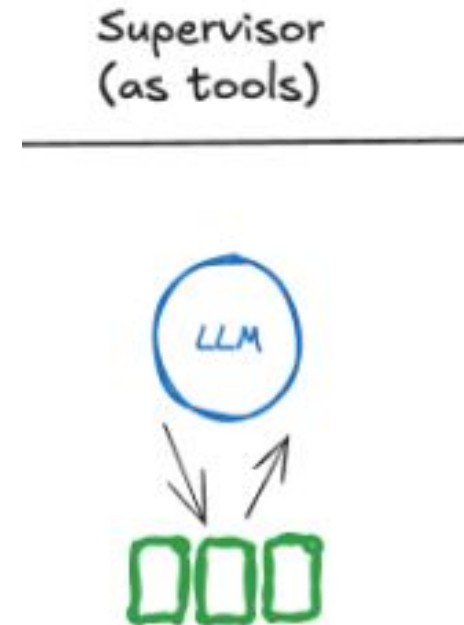


Arquitetura com Supervisor via *Tool-Calling*

- ❑ Variante do modelo Supervisor em que os agentes são tratados como ferramentas (*tools*).
- ❑ O agente supervisor decide qual ferramenta chamar e quais argumentos passar.

Exemplo:

- ❑ Um sistema de suporte ao cliente em que o agente supervisor escolhe ferramentas para verificar estoque, processar pagamentos e responder dúvidas técnicas.



Arquitetura com Supervisor via *Tool-Calling*

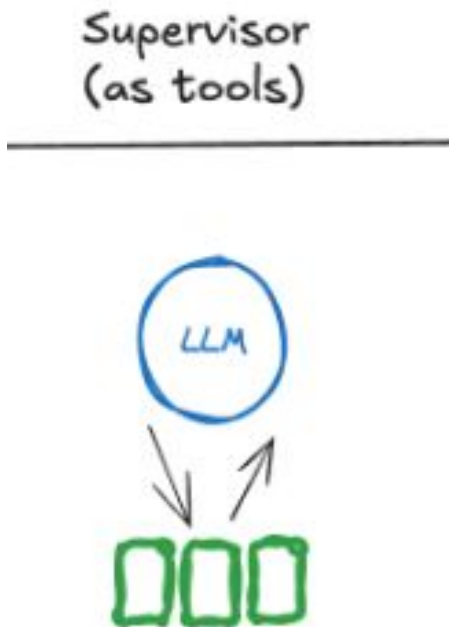
- ❑ Variante do modelo Supervisor em que os agentes são tratados como ferramentas (*tools*).
- ❑ O agente supervisor decide qual ferramenta chamar e quais argumentos passar.

Exemplo:

- ❑ Um sistema de suporte ao cliente em que o agente supervisor escolhe ferramentas para verificar estoque, processar pagamentos e responder dúvidas técnicas.

✓ **Vantagem:** Redução da complexidade da comunicação entre agentes.

✗ **Desafio:** Pode ser menos flexível que outras abordagens.

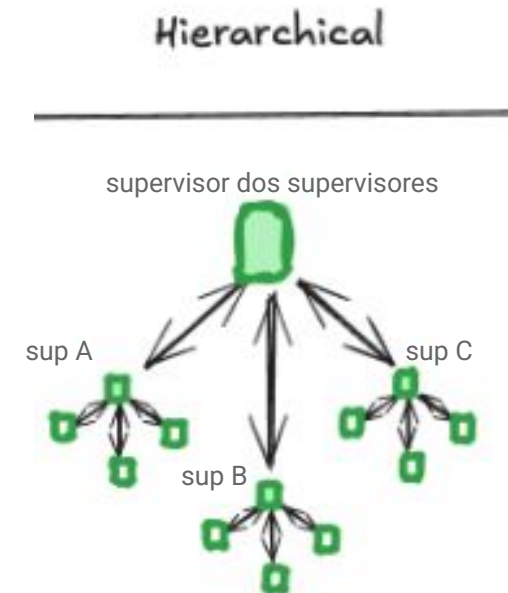


Arquitetura Hierárquica (*Hierarchical*)

- ❑ Expansão da arquitetura Supervisor
- ❑ Há múltiplos supervisores gerenciando equipes de agentes.
- ❑ Agentes podem ser organizados em níveis, facilitando a especialização.

Exemplo:

- ❑ Um assistente de IA para e-commerce em que há um supervisor principal que gerencia equipes de agentes para recomendações, pagamentos e logística.



Arquitetura Hierárquica (*Hierarchical*)

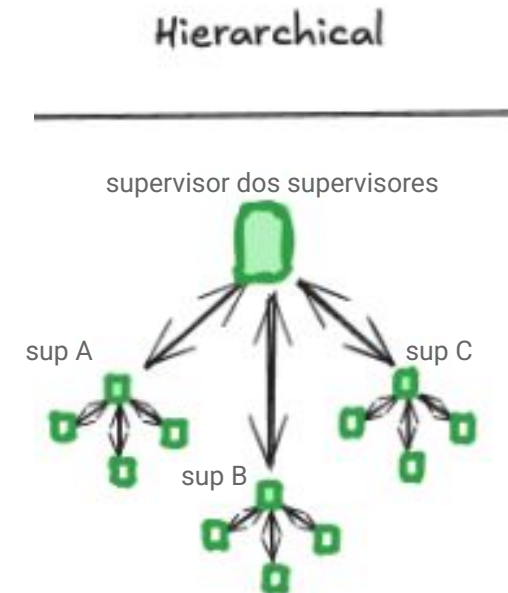
- ❑ Expansão da arquitetura Supervisor
- ❑ Há múltiplos supervisores gerenciando equipes de agentes.
- ❑ Agentes podem ser organizados em níveis, facilitando a especialização.

Exemplo:

- ❑ Um assistente de IA para e-commerce em que há um supervisor principal que gerencia equipes de agentes para recomendações, pagamentos e logística.

✅ **Vantagem:** Melhor organização e escalabilidade.

❌ **Desafio:** Pode ser complexo definir regras de comunicação entre supervisores.

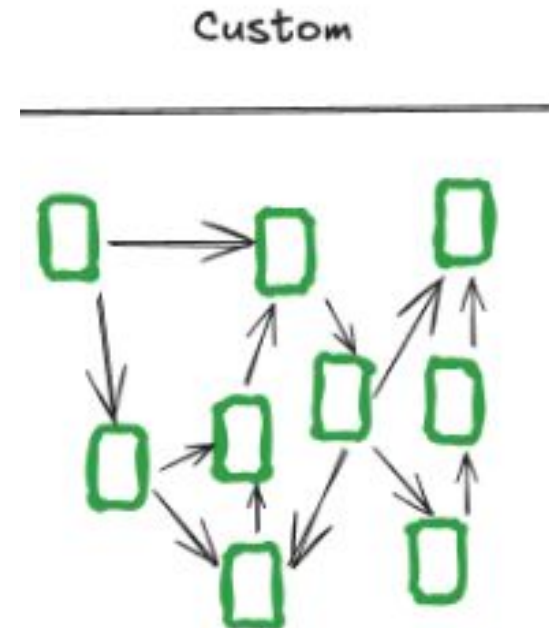


Arquitetura Customizada (*Custom Workflow*)

- ❑ O fluxo de comunicação entre agentes é definido manualmente para atender necessidades específicas.
- ❑ Algumas conexões são fixas, enquanto outras são dinâmicas.

Exemplo:

- ❑ Um pipeline de análise de dados em que cada agente executa um processamento específico antes de passar para o próximo.



Arquitetura Customizada (*Custom Workflow*)

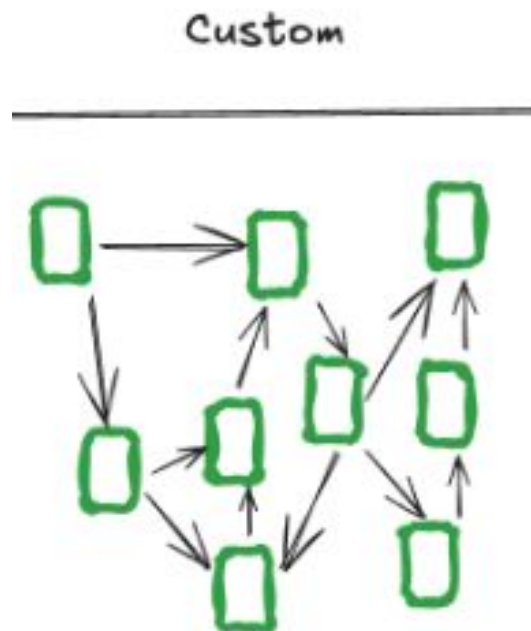
- ❑ O fluxo de comunicação entre agentes é definido manualmente para atender necessidades específicas.
- ❑ Algumas conexões são fixas, enquanto outras são dinâmicas.

Exemplo:

- ❑ Um pipeline de análise de dados em que cada agente executa um processamento específico antes de passar para o próximo.

✅ **Vantagem:** Flexibilidade total na definição do fluxo de trabalho.

❌ **Desafio:** Exige mais esforço para projetar e manter.

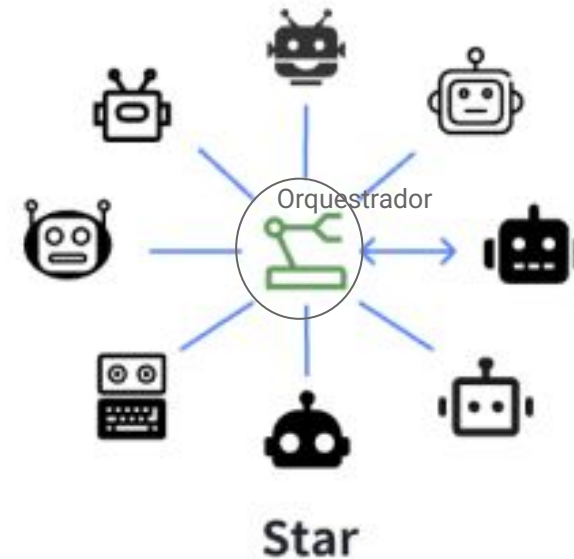


Arquitetura em Estrela (*Star Architecture*)

- ❑ Todos os agentes se conectam a um único nó central, que gerencia a comunicação.
- ❑ Funciona bem quando há um agente coordenador (orquestrador) que centraliza as decisões.

Exemplo:

- ❑ Um sistema de atendimento em que um agente central recebe as requisições dos clientes e distribui para especialistas conforme necessário.



Arquitetura em Estrela (*Star Architecture*)

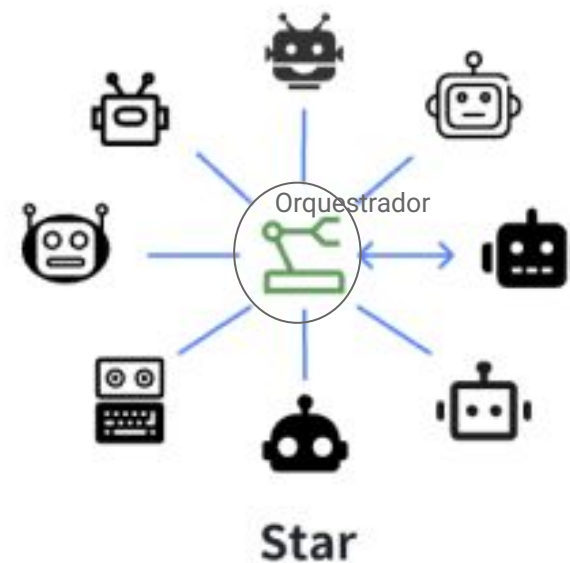
- ❑ Todos os agentes se conectam a um único nó central, que gerencia a comunicação.
- ❑ Funciona bem quando há um agente coordenador (orquestrador) que centraliza as decisões.

Exemplo:

- ❑ Um sistema de atendimento em que um agente central recebe as requisições dos clientes e distribui para especialistas conforme necessário.

✅ **Vantagem:** Fácil de monitorar e gerenciar.

❌ **Desafio:** O nó central pode se tornar um ponto único de falha.

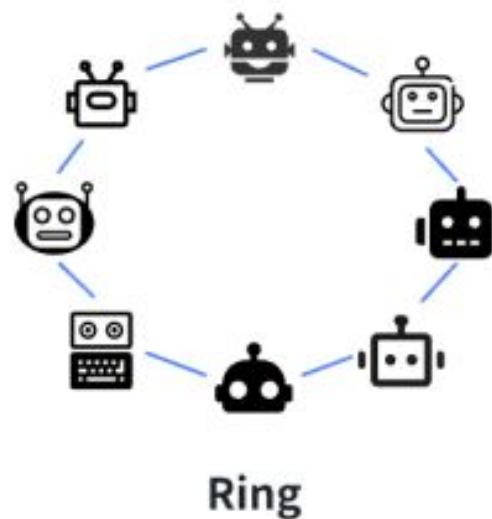


Arquitetura em Anel (*Ring Architecture*)

- ❑ Os agentes estão conectados em um ciclo (pipeline bem definido)
- ❑ Cada um se comunica diretamente com o próximo.
- ❑ A comunicação segue um fluxo pré-definido.

Exemplo:

- ❑ Um sistema de revisão de documentos em que cada agente adiciona sugestões antes de passar para o próximo.



Arquitetura em Anel (*Ring Architecture*)

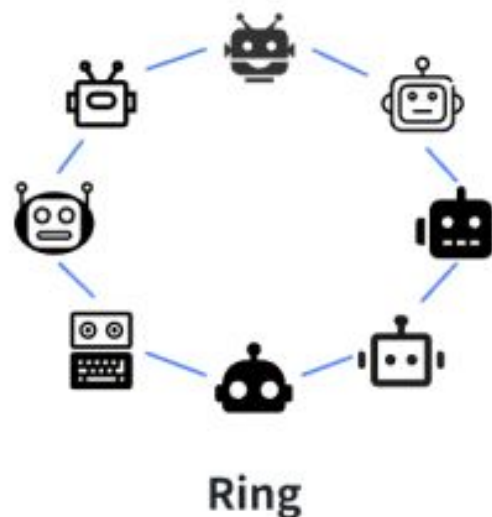
- ❑ Os agentes estão conectados em um ciclo (pipeline bem definido)
- ❑ Cada um se comunica diretamente com o próximo.
- ❑ A comunicação segue um fluxo pré-definido.

Exemplo:

- ❑ Um sistema de revisão de documentos em que cada agente adiciona sugestões antes de passar para o próximo.

✓ **Vantagem:** Comunicação eficiente e previsível.

✗ **Desafio:** Pode ser rígida para certos tipos de problemas.

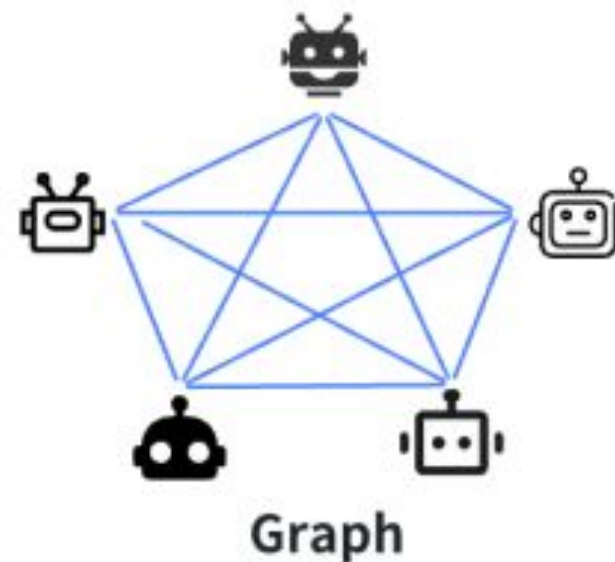


Arquitetura em Grafo (*Graph Architecture*)

- ❑ Os agentes formam um grafo, permitindo comunicações mais dinâmicas.
- ❑ Pode combinar elementos de rede e supervisão.

Exemplo:

- ❑ Diferentes agentes lidam com partes de um processo e podem se comunicar conforme necessário.



Arquitetura em Grafo (*Graph Architecture*)

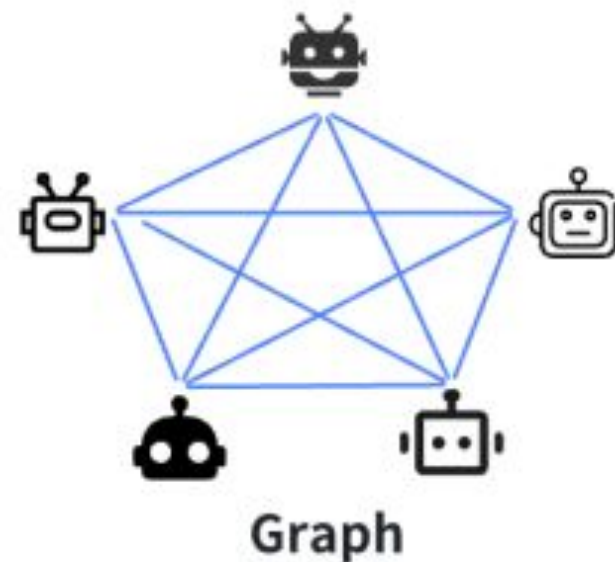
- ❑ Os agentes formam um grafo, permitindo comunicações mais dinâmicas.
- ❑ Pode combinar elementos de rede e supervisão.

Exemplo:

- ❑ Diferentes agentes lidam com partes de um processo e podem se comunicar conforme necessário.

✓ **Vantagem:** Maior flexibilidade e eficiência.

✗ **Desafio:** Exige um planejamento mais detalhado da comunicação.

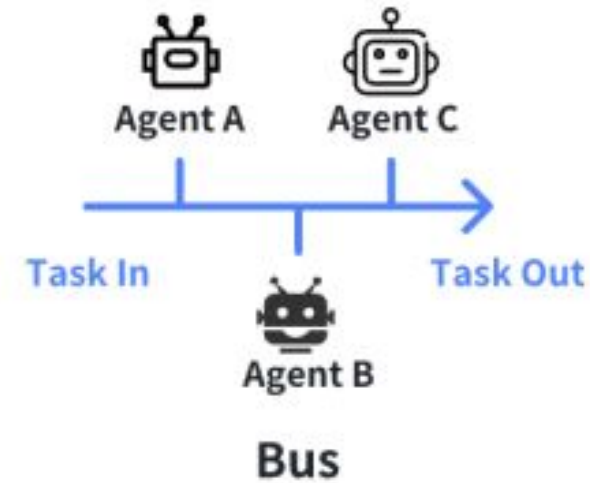


Arquitetura em Barramento (*Bus Architecture*)

- ❑ Os agentes se comunicam através de um barramento central que gerencia mensagens entre eles.
- ❑ Reduz a necessidade de conexões diretas entre agentes.

Exemplo:

- ❑ Um sistema de processamento de pedidos em que diferentes agentes atuam de forma independente, mas compartilham informações pelo barramento.



Arquitetura em Barramento (*Bus Architecture*)

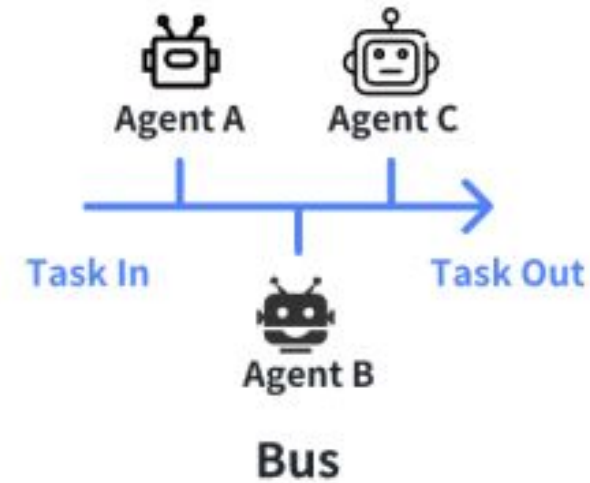
- ❑ Os agentes se comunicam através de um barramento central que gerencia mensagens entre eles.
- ❑ Reduz a necessidade de conexões diretas entre agentes.

Exemplo:

- ❑ Um sistema de processamento de pedidos em que diferentes agentes atuam de forma independente, mas compartilham informações pelo barramento.

✓ **Vantagem:** Boa escalabilidade e flexibilidade.

✗ **Desafio:** Exige um barramento robusto para evitar falhas.



Comparação – Arquiteturas Topológicas

Arquitetura	Flexibilidade	Escalabilidade	Coordenação	Complexidade
Star	Média	Média	Alta	Baixa
Ring	Baixa	Média	Muito alta	Muito baixa
Bus	Média	Alta	Média	Média
Graph	Muito alta	Muito alta	Baixa	Alta

Padrões arquiteturais “modernos”

❏ Planner → Executor

Padrões arquiteturais “modernos”

- ❑ Planner → Executor
- ❑ Planner → Researcher → Reasoner → Executor → Reviewer

Padrões arquiteturais “modernos”

- ❑ Planner → Executor
- ❑ Planner → Researcher → Reasoner → Executor → Reviewer
- ❑ Planner → Parallel Specialists → Aggregator

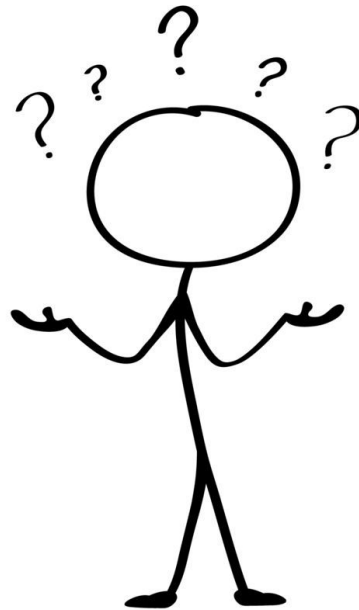
Padrões arquiteturais “modernos”

- ❑ Planner → Executor
- ❑ Planner → Researcher → Reasoner → Executor → Reviewer
- ❑ Planner → Parallel Specialists → Aggregator
- ❑ Debate: Agente A → Agente B → Juiz

Padrões arquiteturais “modernos”

- ❑ Planner → Executor
- ❑ Planner → Researcher → Reasoner → Executor → Reviewer
- ❑ Planner → Parallel Specialists → Aggregator
- ❑ Debate: Agente A → Agente B → Juiz
- ❑ Reflection: Agente → Critic → Nova resposta

Como Escolher a Melhor Arquitetura?



Critérios para Escolher uma Arquitetura

- ❑ **Complexidade:** O sistema precisa ser simples ou escalável?
- ❑ **Latência:** Precisa de respostas rápidas ou pode haver demora na comunicação?
- ❑ **Resiliência:** Como o sistema lida com falhas de agentes?
- ❑ **Facilidade de implementação:** Qual arquitetura é mais fácil de desenvolver e manter?

Trade offs arquiteturas

❏ Quanto maior o número de agentes:

- ↑ especialização
- ↑ qualidade
- ↑ paralelismo
- ↑ modularidade

❏ Efeito:

- ↑ custo
- ↑ latência
- ↑ comunicação
- ↑ necessidade de governança

Síntese

- ❑ Sistemas multiagentes dividem problemas complexos em agentes especializados
- ❑ A escolha da arquitetura impacta diretamente desempenho, escalabilidade, robustez e custo.
- ❑ Não existe uma arquitetura universalmente melhor; a escolha depende dos requisitos da aplicação.
- ❑ Arquiteturas como Supervisor, Hierarchical e Graph são amplamente utilizadas em sistemas Agentic AI modernos.